

Reviewer Pack — Minimal Modular Symmetry Rank and Circuit Monotone Width Ine...

Entry 2634fcde1ae3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 10:47:38 UTC

Minimal Modular Symmetry Rank and Circuit Monotone Width Inequality

Entry ID: 2634fcde1ae3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 10:47:38 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Modular Symmetry Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every n -ary Boolean function f , the minimal rank of its modular symmetry group $M(f)$ is linearly correlated with its circuit monotone width $w(f)$, such that $\min_{\{g \in M(f)\}} w(g) = \Theta(w(f))$.

Rationale (proposer's reasoning):

Modular symmetry theory provides a framework for analyzing discrete symmetries in algebraic structures, and its application to Boolean functions could potentially reveal underlying patterns related to computational complexity. If the minimal rank of the modular symmetry group is correlated with circuit monotone width, it may indicate a deeper connection between algebraic symmetries and the hardness of computation.

Taxonomy category: Modular_Symmetry_Theory × Boolean_Circuit_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2608f225697f5b58

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given n -ary Boolean function f , if the minimum rank of its modular symmetry group $M(f)$ is within a factor of 2 from its circuit monotone width $w(f)$, and no seed produces a discrepancy greater than 10 between $\min_{\{g \in M(f)\}} w(g)$ and $w(f)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "modular symmetry rank" AND "circuit monotone width" - "Algebraic Combinatorics" AND "Boolean Circuit Complexity" AND "symmetry group" - "minimal rank of modular symmetry" AND Boolean functions AND circuit complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def circuit_monotone_width(f):
        n = int(math.log2(len(f)))
        width = 0
        for i in range(n):
            count = sum(1 for x in f if x & (1 << i) == 0)
            width = max(width, count)
        return width

    def modular_symmetry_group(f):
        n = int(math.log2(len(f)))
        symmetries = set()
        for perm in itertools.permutations(range(n)):
            g = [f[perm[i]] for i in range(n)]
            if g not in symmetries:
                symmetries.add(tuple(g))
        return symmetries

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_random_boolean_function(n)
        w_f = circuit_monotone_width(f)
        M_f = modular_symmetry_group(f)
```

```

min_w_g = min(circuit_monotone_width(g) for g in M_f)

if len(M_f) == 0 or min_w_g == 0:
    continue

results.append({
    "n": n,
    "w_f": w_f,
    "min_w_g": min_w_g
})

if not results:
    return {
        "metric_name": "min_w_g_over_w_f",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

min_w_g_over_w_f = sum(result["min_w_g"] / result["w_f"] for result in results) / len(results)
instances_tested = len(results)
n_max = max(result["n"] for result in results)

return {
    "metric_name": "min_w_g_over_w_f",
    "metric_value": min_w_g_over_w_f,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": 0.5 <= min_w_g_over_w_f <= 2.0,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all("metric_value" not in result or result["metric_value"] is None for result in results):
        print("RESULT: INCONCLUSIVE mapping_undefined")
    else:
        mean_metric = sum(result["metric_value"] for result in results) / len(results)
        std_metric = math.sqrt(sum((result["metric_value"] - mean_metric)**2 for result in results) / len(results))
        support_fraction = sum(1 for result in results if 0.5 <= result["metric_value"] <= 2.0) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
        else:
            first_failing_seed = next(seed for seed, result in zip(seeds, results) if not (0.5 <= result["metric_value"] <= 2.0))
            print(f"RESULT: FALSIFIED counterexample=\"min_w_g_over_w_f out of range\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad1e01e6.py", line 88, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad1e01e6.py", line 47, in run_trial
    M_f = modular_symmetry_group(f)
         ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad1e01e6.py", line 37, in modular_symme
    if g not in symmetries:
       ~~~~~~
```

TypeError: unhashable type: 'list'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying whether the conjecture's support conditions are met. | next: Investigate the cause of the crash and attempt to run the test again to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17158
2	preregistration	ollama_remote	glm4:latest	0	0	18824
3	novelty	ollama_remote	glm4:latest	0	0	8689
4	novelty	ollama_remote	glm4:latest	0	0	8759
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25973
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12926
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10702
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12821
9	judge	ollama_remote	glm4:latest	0	0	24555

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140408 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/2634fcde1ae3.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2634fcde1ae3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2634fcde1ae3.tar.gz` (if generated)